

第九节 MXMACA程序优化理论

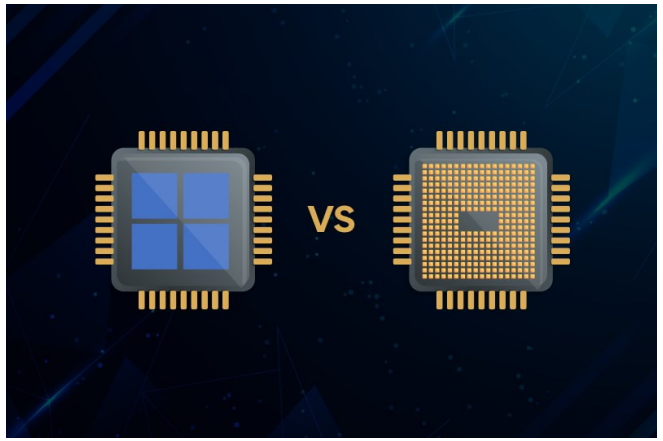
——《高级计算机体系结构》— GPU架构与编程

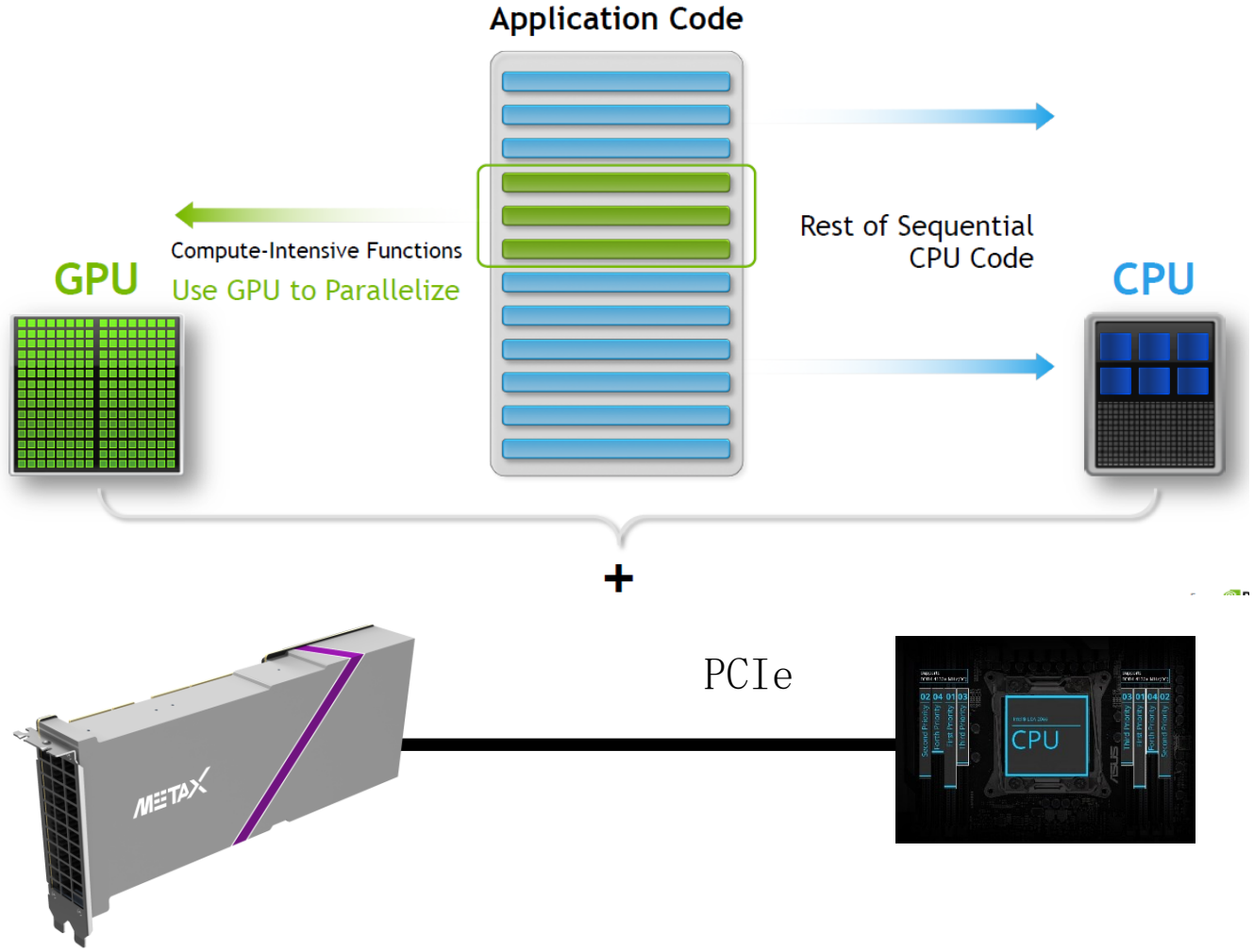
GPU加速异构计算系统

用语介绍：

Host：CPU及主存(host memory)

Device：GPU及显存(device memory)





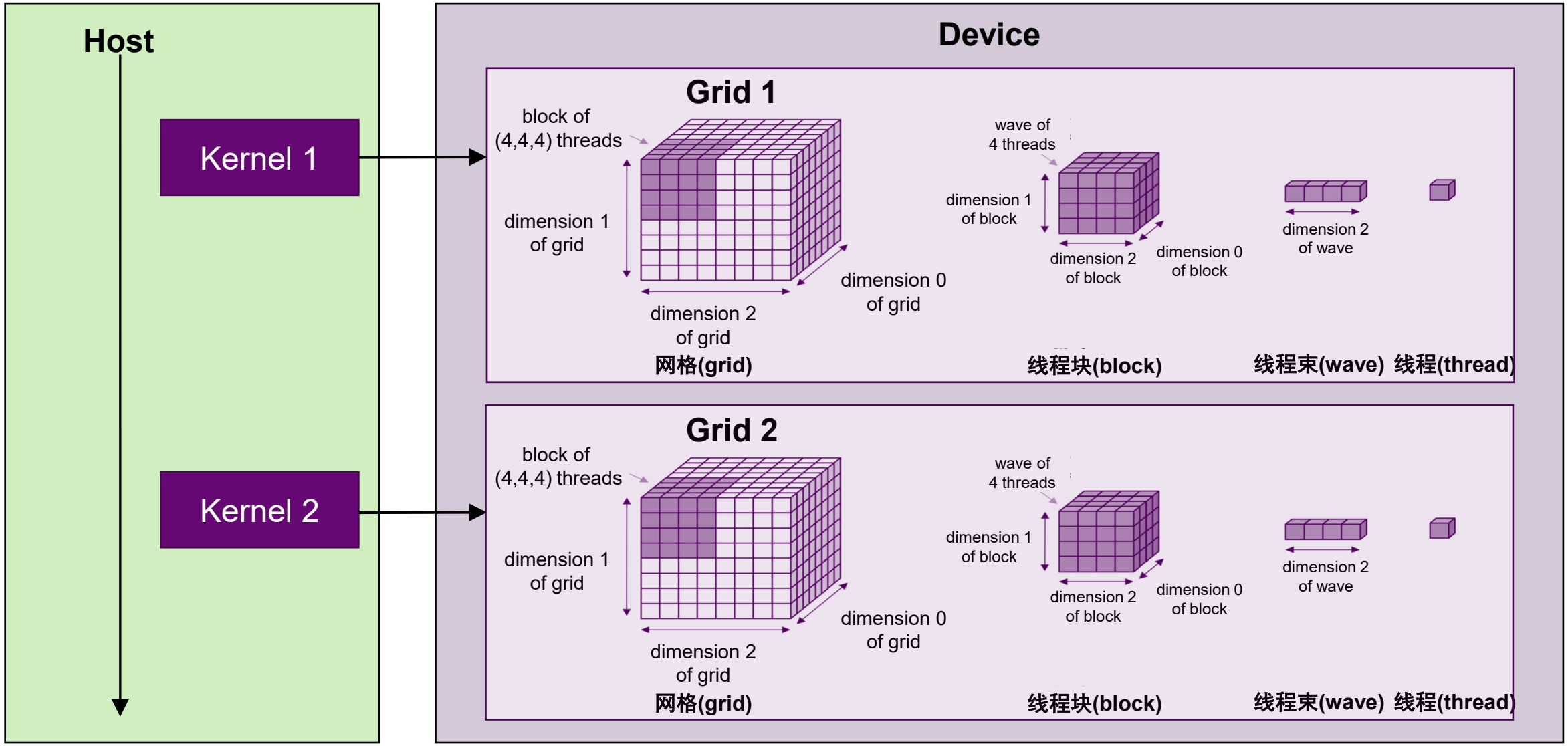
| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |



MXMACA程序优化基础理论

| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

MXMACA应用开发的性能：如何提升GPU利用效率？



| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

Thread Coarsening

Thread Coarsening

- 线程粗化意味着减少活跃线程的数量，但每个线程处理更多的数据。
- 这样做的目的是减少线程调度和同步的开销，同时更有效地利用GPU的计算资源。
- 例如，如果原本每个线程处理一个元素，线程粗化后，一个线程可能处理四个或更多的元素。

示例： Vector addition with thread coarsening

- https://gitee.com/Inkstoneydz/mxmaca-courses-zju/tree/master/demo/08_access/02

| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

Optimize grid dimensions based on device properties

Occupancy 示例

- https://gitee.com/Inkstoneydz/mxmaca-courses-zju/tree/master/demo/08_access/04

Block大小的选择

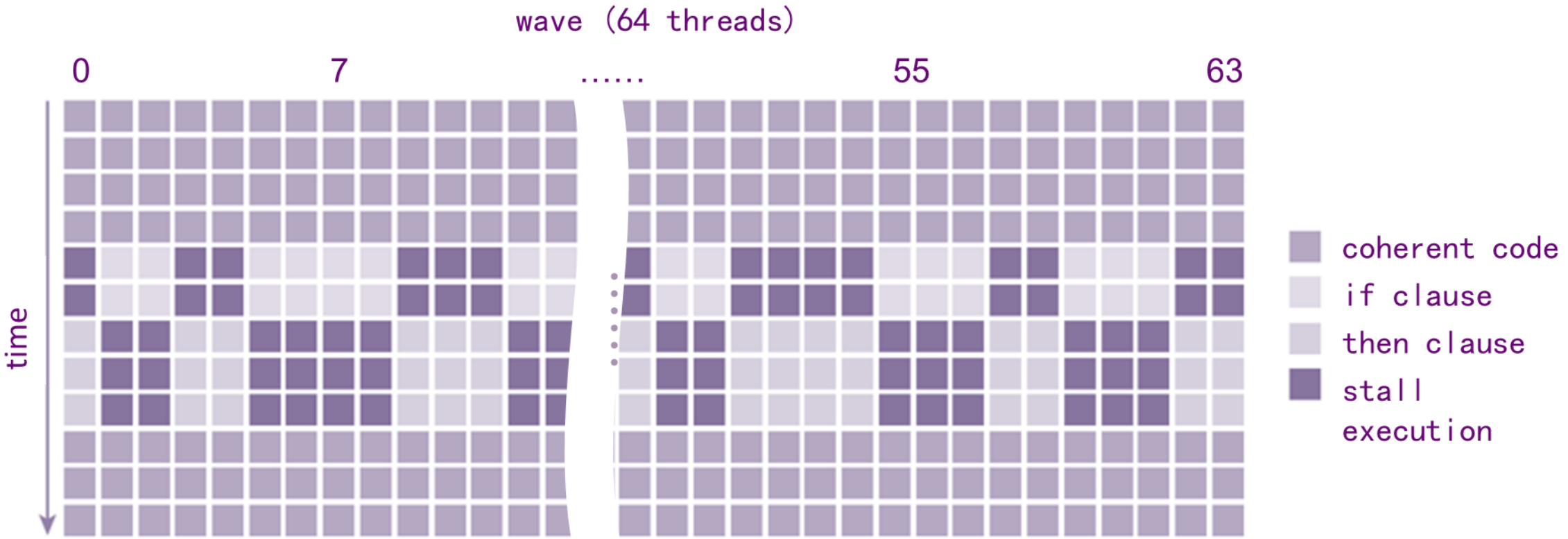
- mcOccupancyMaxPotentialBlockSize：它可以计算出在单个AP占用率最大时blockSize的大小，可以根据此值配置blockSize。这样你就可以不去关心各种硬件资源的限制（如果你觉得这是一件很麻烦的事情）。
- 可以在/opt/maca/include/mcr/mc_runtime_api.h搜“Occupancy”了解更多Occupancy相关的API

| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

Wave Divergence

➢线程束分化示例： Vector addition with thread coarsening

➤ https://gitee.com/lnkstoneydz/mxmaca-courses-zju/tree/master/demo/08_access/03



| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

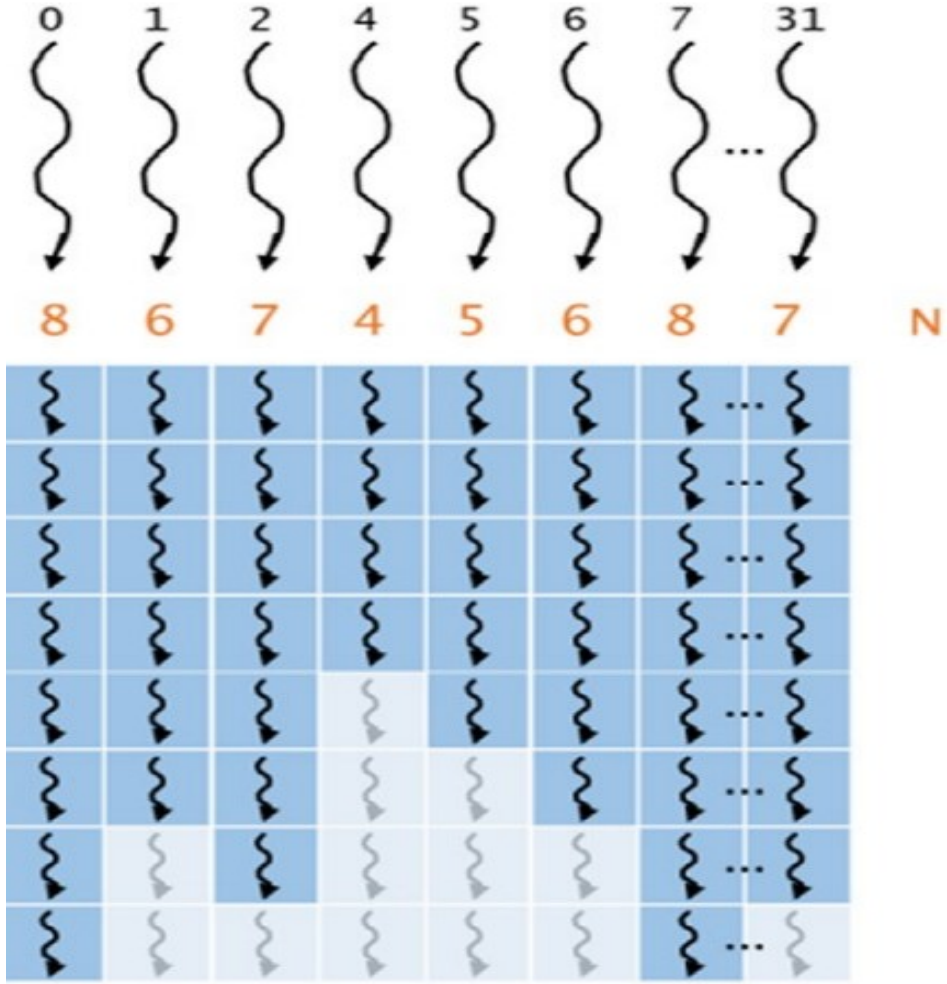
线程束何时执行完毕？

- 带有变量上界的循环也可能会导致线程束分化

```
N = a[threadIdx.x];
for(i = 0; i < N; ++i) {

    A

}
```



| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

MXMACA内存模型

图6-2 GPU内存的层次结构

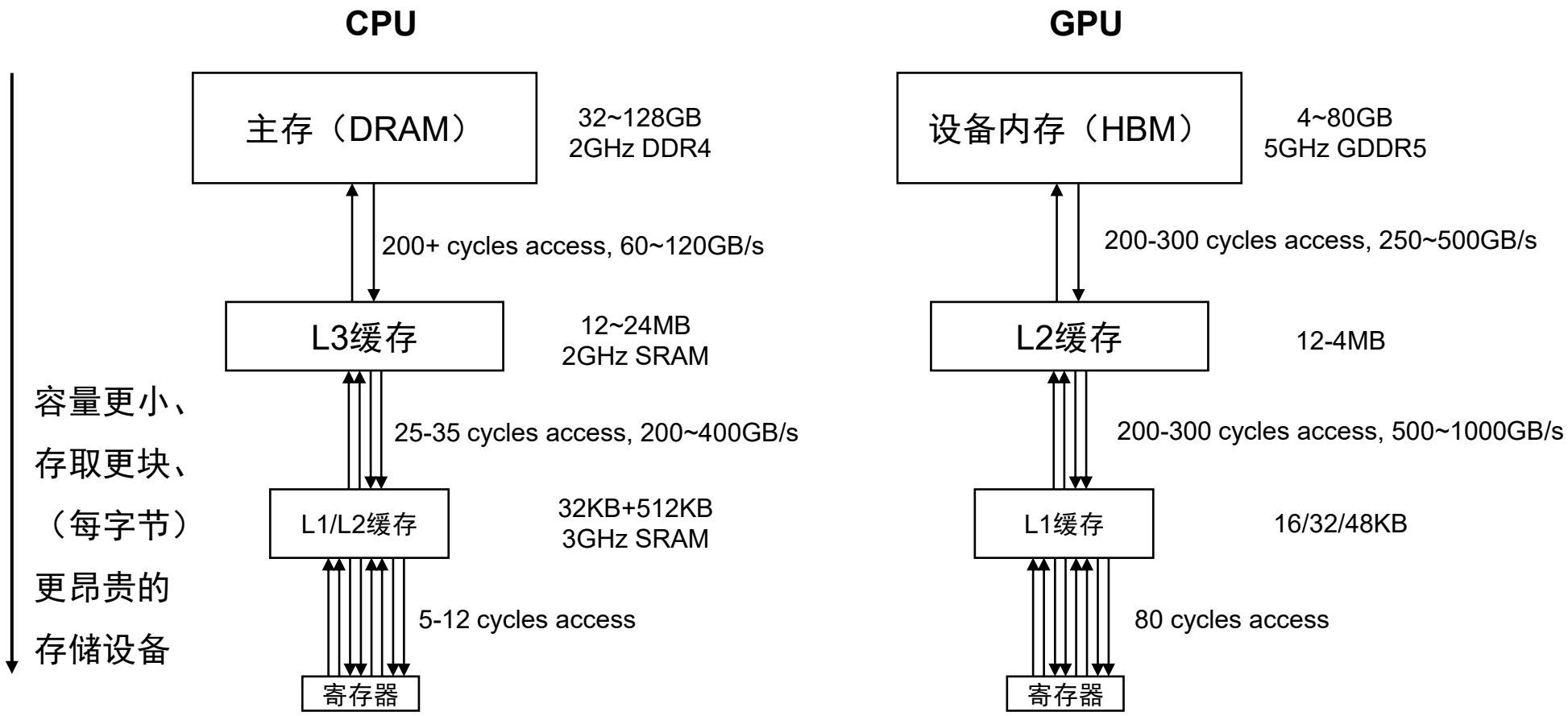


图6-2

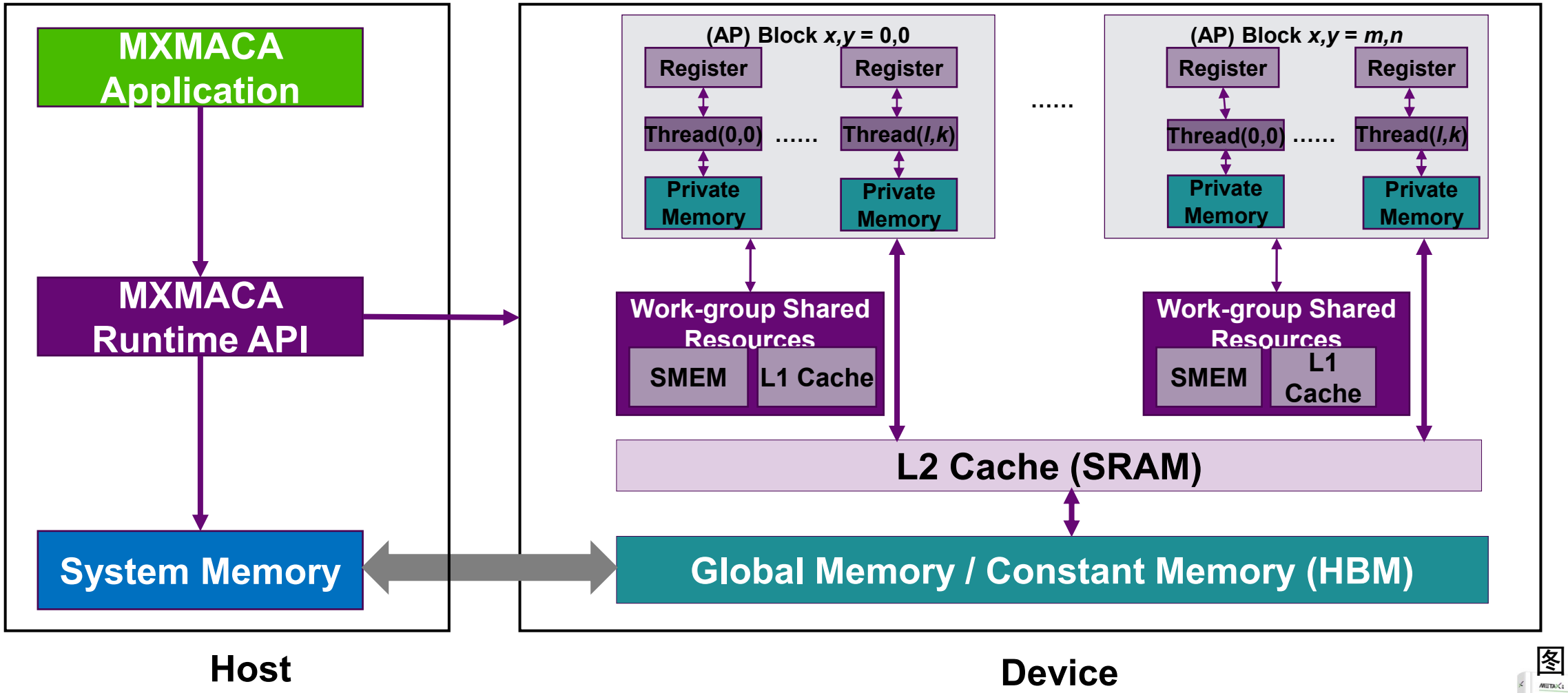


与CPU相比，GPU的带宽更高、延迟更大、缓存更少。此外，GPU编程模型会将内存层次结构呈现给程序员，给程序员显示控制GPU的API，而CPU则一般不给出控制内存层次结构的API。

| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

MXMACA内存模型

图6-6 MXMACA编程的内存层次模型



GPU的可编程存储单元包括全局内存、常量内存、共享内存、私有内存和寄存器等，
GPU的不可编程存储单元则包括一级缓存、二级缓存等。

图6-6



| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

MXMACA内存模型

表6-3 GPU设备存储器的特征

表6-2 MXMACA变量和类型修饰符

修饰符	变量名称	存储器位置	作用域
n/a	float var	寄存器	线程
n/a	float var[100]	私有内存	线程
__shared__	float var *	共享内存	线程块
__device__	float var *	全局内存	全局
__constant__	float var *	常量内存	全局

Device code can:

- R/W per-thread registers
- R/W per-thread local memory
- R/W per-block shared memory
- R/W per-grid global memory
- Read only per-grid constant memory

Host code can

- Transfer data to/from per grid global and constant memories

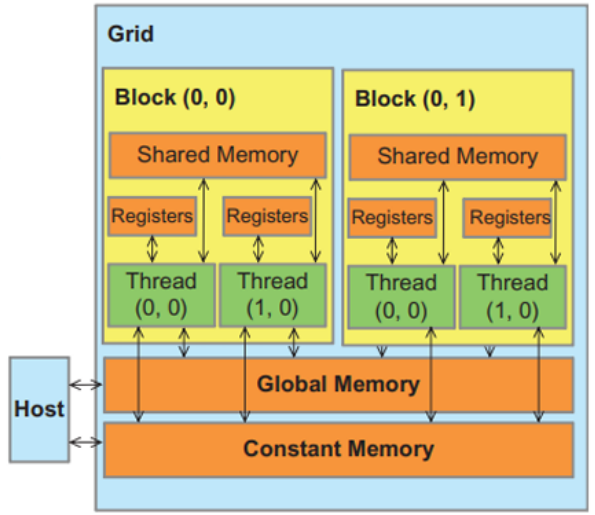


表6-3 GPU设备存储器的特征

存储器	位置	缓存	读写操作	范围	生命周期	速度
寄存器	芯片内	无缓存	读/写	一个线程	与线程生命周期相同	最快
私有内存	芯片外	L1/L2缓存	读/写	一个线程	与线程生命周期相同	较慢
共享内存	芯片内	无缓存	读/写	块内所有线程	与线程块生命周期相同	在无存储体冲突时快
全局内存	芯片外	L1/L2缓存	读/写	所有线程＋主机	与程序生命周期相同	较慢
常量内存	芯片外	L1/L2缓存	读	所有线程＋主机	与程序生命周期相同	较慢

表6-3



| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |



Roofline模型

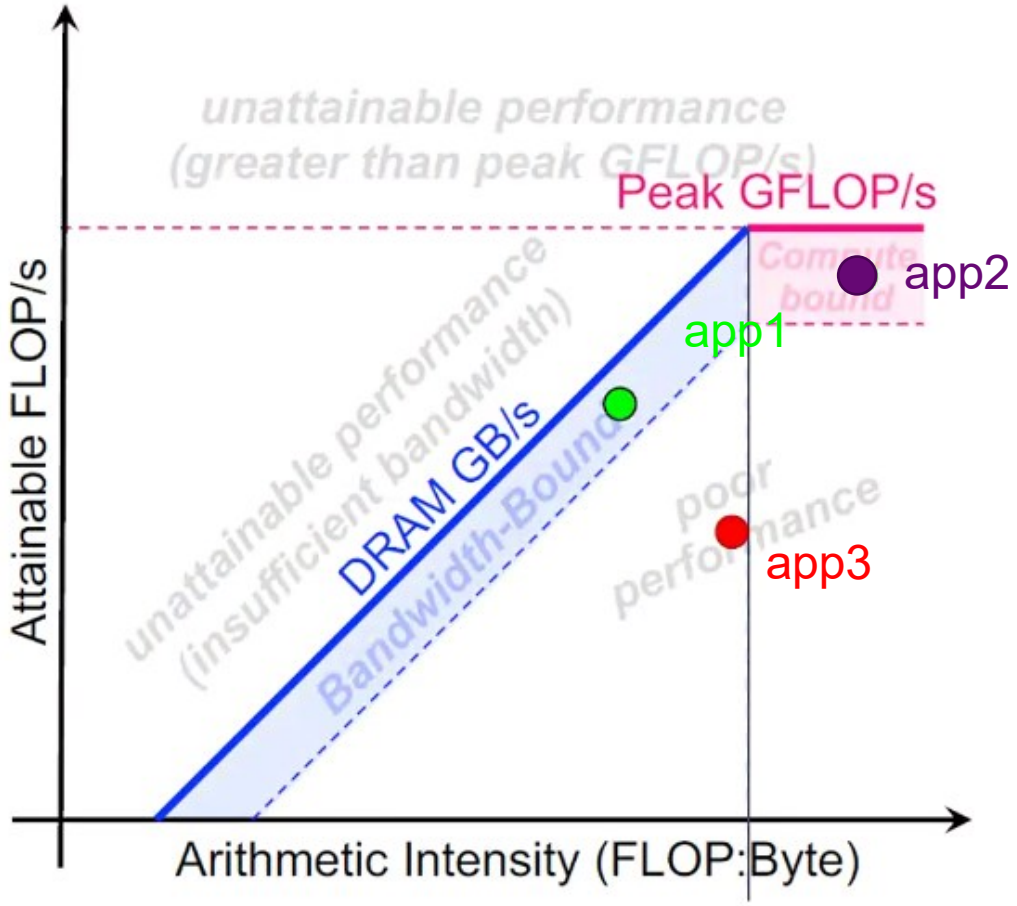
| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

MXMACA应用程序的Roofline模型

$$GFLOP/s = \min \begin{cases} \text{Peak GFLOP/s} \\ \text{AI} * \text{Peak GB/s} \end{cases}$$

Where AI = #FLOP / #Byte
(Arithmetic Intensity, 特定应用的算术密度)

- AI为横坐标，画在Log-Log坐标系中得到屋檐
- 算力带宽比 Peak GFLOP/s / Peak GB/s
- 给定软件，可以分析其在给定硬件上的计算访存比
- 计算访存比 < 算力带宽比，该软件性能受限于带宽
- 计算访存比 > 算力带宽比，该软件性能受限与算力

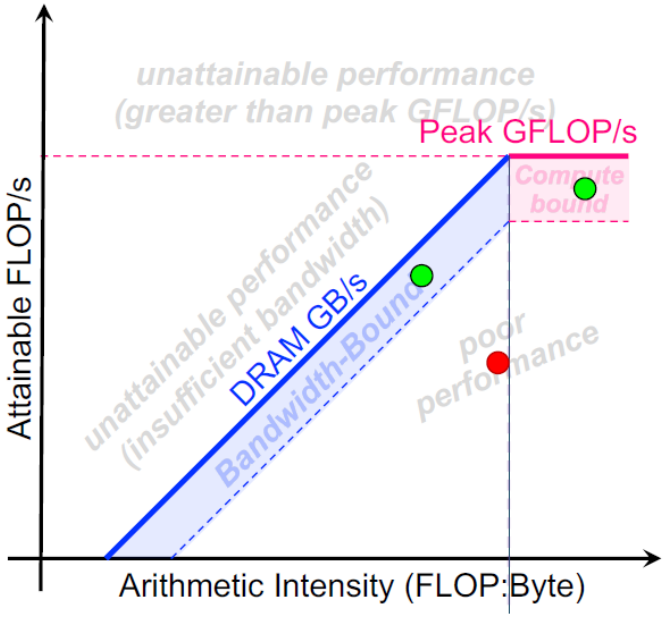


| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

复杂系统的建模来自简洁的数学模型

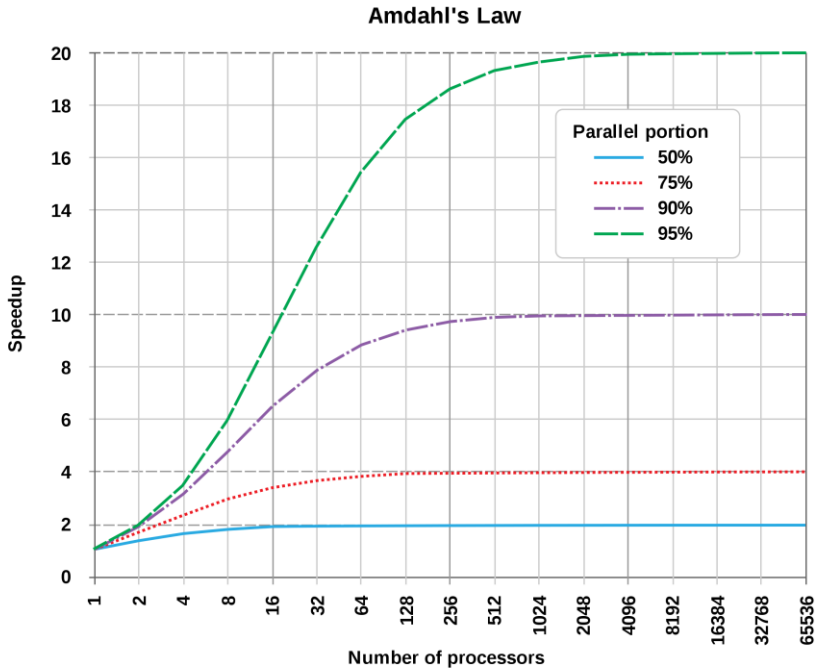
Roofline Model

$$\text{GFLOP/s} = \min \begin{cases} \text{Peak GFLOP/s} \\ \text{AI} * \text{Peak GB/s} \end{cases}$$



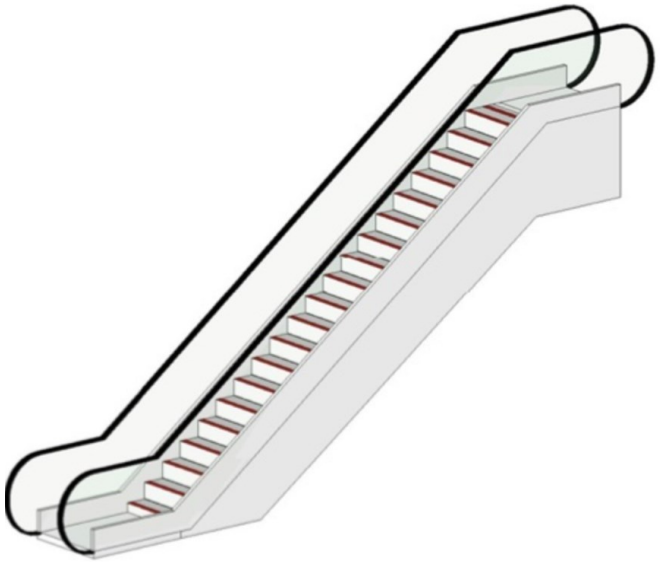
Strong (weak) scaling law

$$S_p \le \frac{1}{(1-f)+\frac{f}{p}}$$



Little's Law

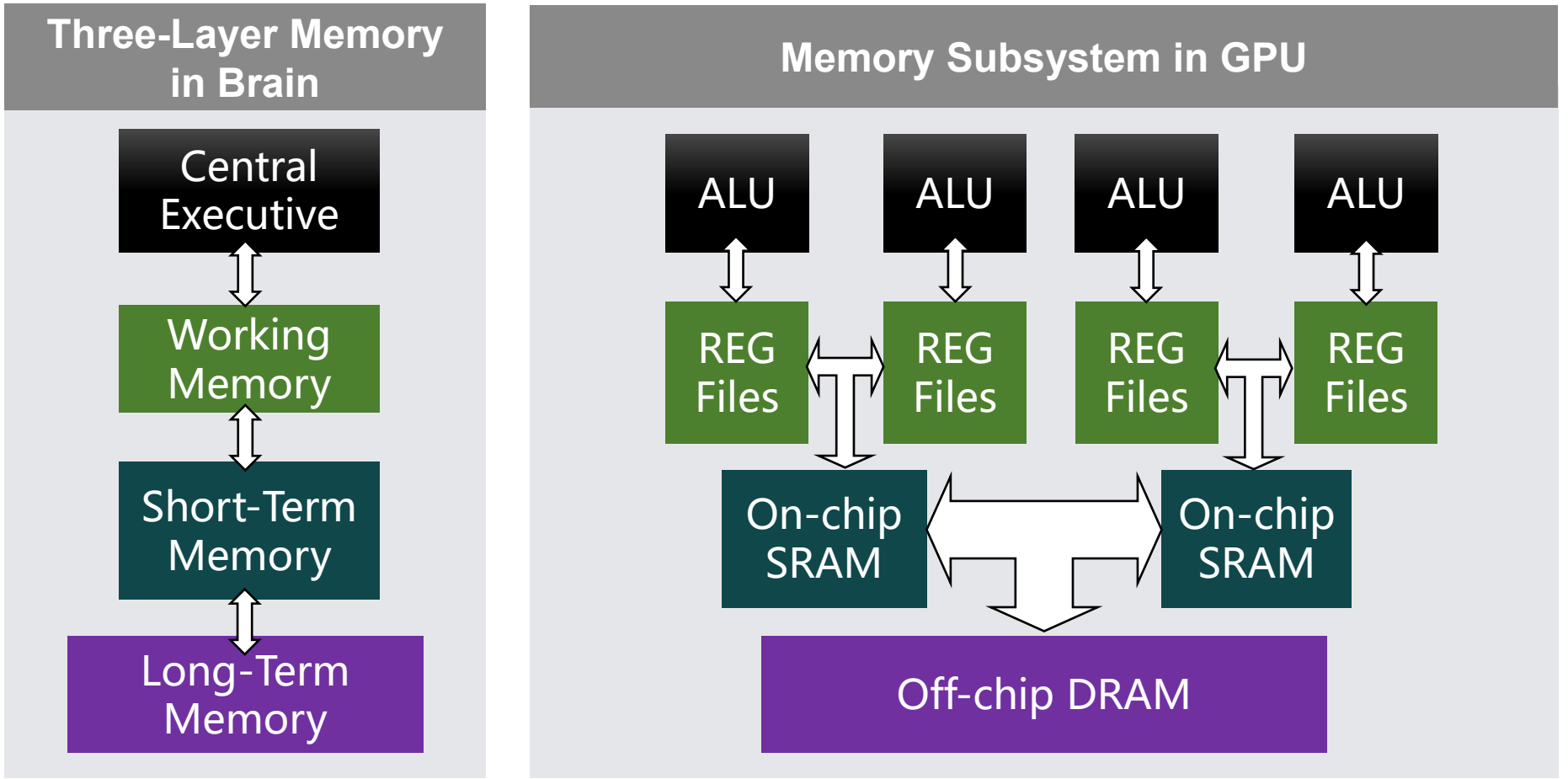
$$R_x = \min \begin{cases} R_max \\ \frac{1}{L} \times T_e \end{cases}$$



| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

GPU Memory Subsystem

› GPU Memory Subsystem can be analogized to Three-Layer Memory in Cognitive Science



Arithmetic Logic Unit (ALU)
- Scalar + Vector + Matrix Processing Elements

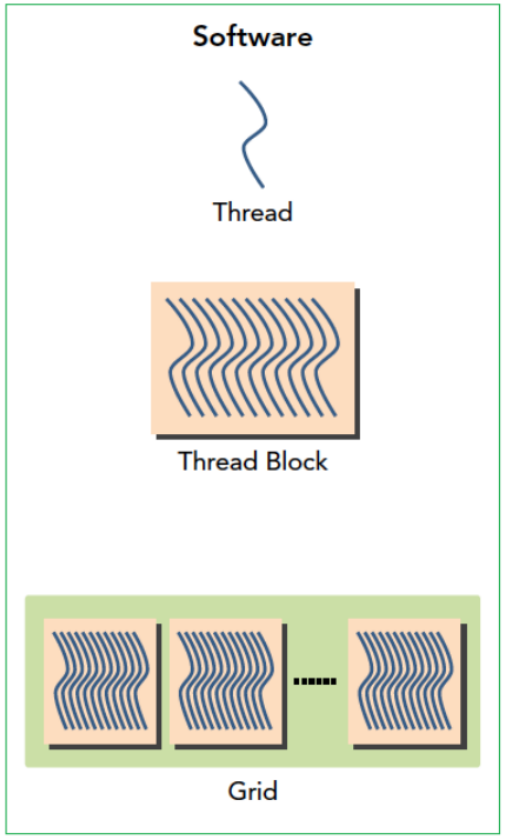
On-chip SRAM
- Smaller
- Faster
- cache + shared memory

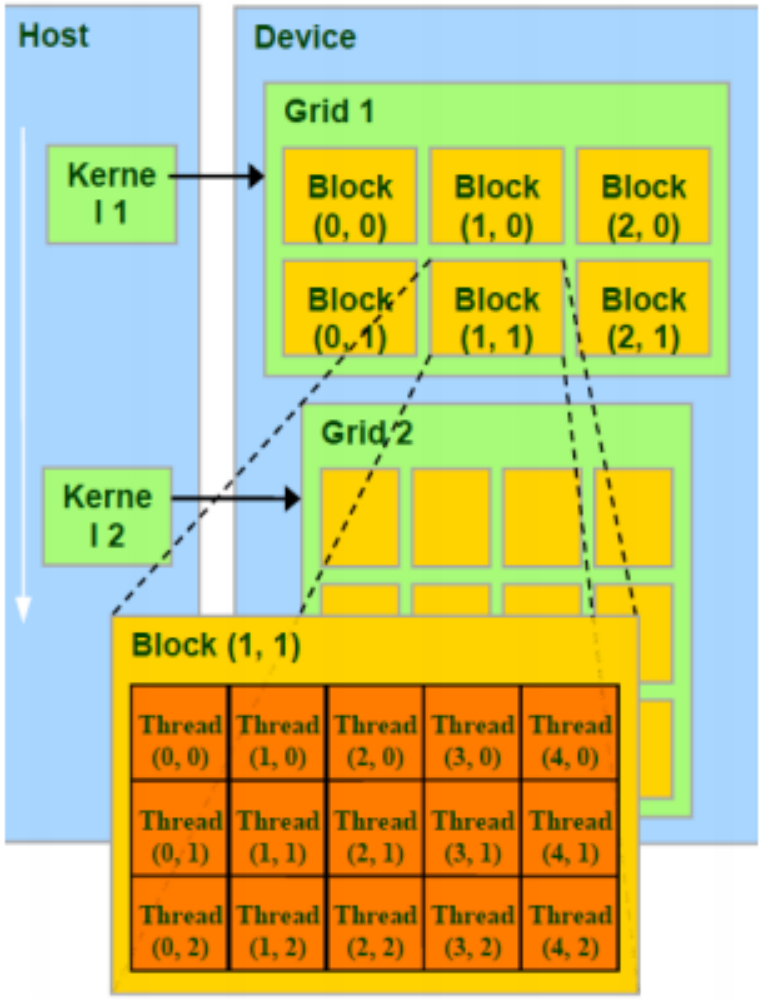
Off-chip DRAM
- Larger
- Slower

| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

Recap: GPU编程三级抽象Thread – Block - Grid

- **Thread**
  - 最小的计算单元
  - 独占寄存器堆
- **Warp/Wave: 硬件SIMD执行**
- **Block**
  - Thread组成的三维结构
  - 共享shared memory
- **Grid**
  - Block组成的三维结构
  - 共享global memory





| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

关注沐曦

企业官网

http://www.metax-tech.com/

官方微信公众号

官方视频号

B站官方频道

开发者社区

| 沐曦内部资料 | MetaX Confidential | All Rights Reserved |

DISCLAIMER & ATTRIBUTION

DISCLAIMER

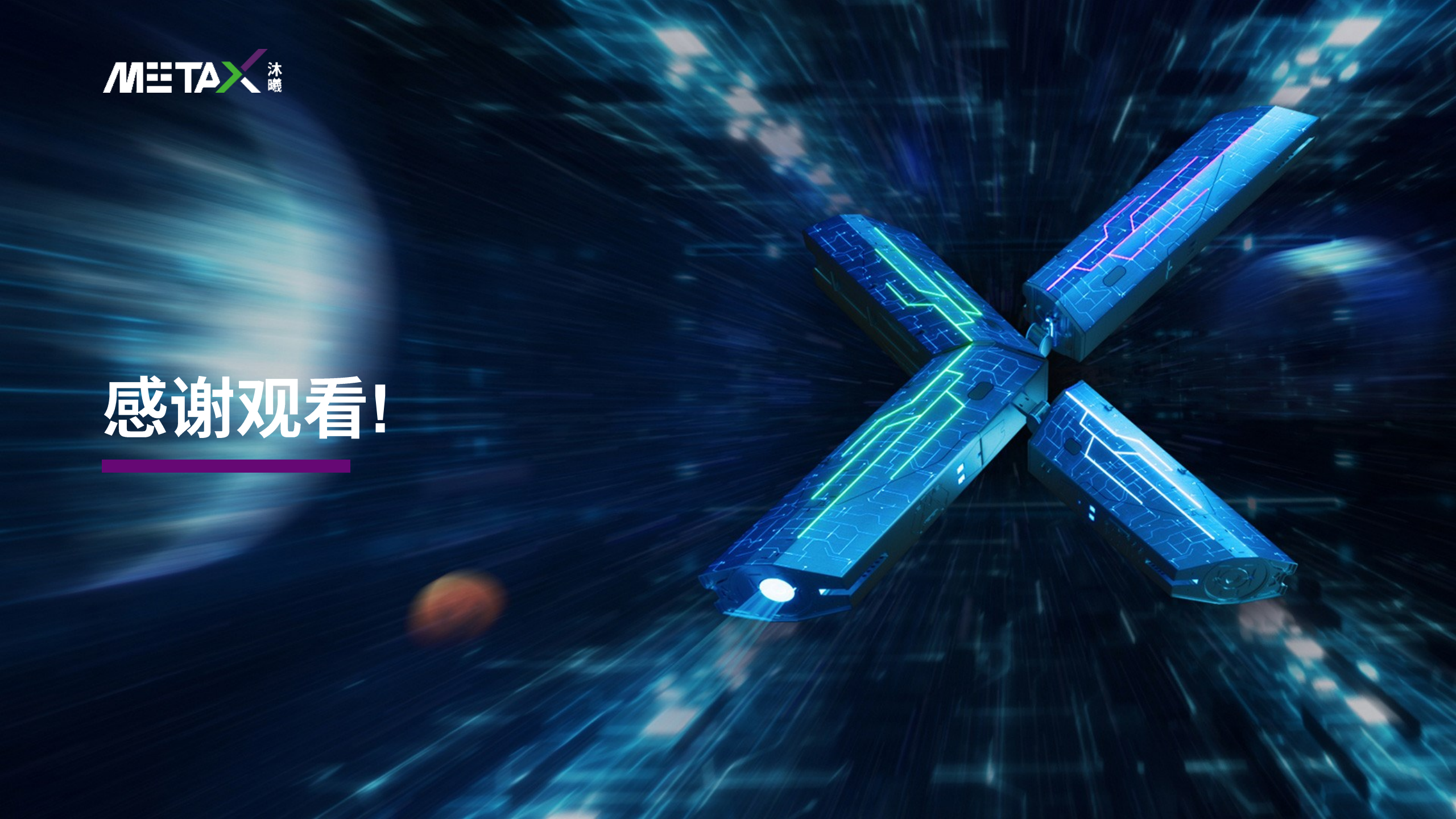
The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions and typographical errors.

The information contained herein is subject to change and may be rendered inaccurately for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between different manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. MetaX assumes no obligation to update or otherwise correct or revise this information. However, MetaX reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of MetaX to notify any person of such revisions or changes.

ATTRIBUTION

©2020 MetaX Integrated (Shanghai) Co., Ltd. All rights reserved. MetaX, 沐曦, the MetaX logo and combinations thereof are trademarks of MetaX Integrated (Shanghai) Co., Ltd. in the PRC China and/or other jurisdictions. Other names are for informational purposes only and may be trademarks of their respective owners.

http://www.metax-tech.com/


METAX
沐曦
感谢观看!